Phase 1: Project Proposal & Environment Setup

Data Engineering Course | Project: Bank Customer Churn Pipeline

1. Team Members

Full Name	Student ID	Role
[Member 1 full name]	[ID]	Team lead / Docker & PostgreSQL setup
[Member 2 full name]	[ID]	Python ingestion & data cleaning
[Member 3 full name]	[ID]	SQL schema, quality checks & views
[Member 4 full name]	[ID]	Kestra orchestration flows
[Member 5 full name]	[ID]	Dashboard & documentation

Fill in names/IDs and delete unused rows (team size 3-5). Every member must understand the full workflow.

2. Dataset

Name: Bank Customer Churn (European bank customers; widely published on Kaggle as "Churn Modelling").
Link: [paste your download link here, e.g. the Kaggle page you downloaded it from]
Format: CSV (Bank_Customer_Churn.csv) | **Rows:** 10,000 | **Columns:** 14 | **Missing values:** none | **Duplicate rows:** none.
Description: One row per bank customer with demographics, account information and a target flag (Exited) showing whether the customer left the bank. About 20.4% of customers churned, so the dataset is suitable for churn analysis. The 10,000 rows meet the minimum requirement and it is not the NYC Taxi dataset.

Column	Data type	Category	Notes
RowNumber	Integer	ID	Row index; dropped in clean table
CustomerId	Integer	ID	Unique customer key
Surname	Text	Text	2,932 distinct values
CreditScore	Integer	Numeric	350 to 850
Geography	Text	Categorical	France, Germany, Spain
Gender	Text	Categorical	Male, Female
Age	Integer	Numeric	18 to 92
Tenure	Integer	Numeric	0 to 10 years
Balance	Decimal	Numeric	Many zero balances
NumOfProducts	Integer	Numeric/Cat.	1 to 4
HasCrCard	Integer (0/1)	Binary flag	Credit card owner
IsActiveMember	Integer (0/1)	Binary flag	Active customer
EstimatedSalary	Decimal	Numeric	Annual salary estimate
Exited	Integer (0/1)	Target	1 = left the bank

Note: the file has no date column, so questions are based on categories and numeric segments rather than time trends. If the instructor requires a date column, we will add an ingestion timestamp (ingested_at) for pipeline-run analysis.

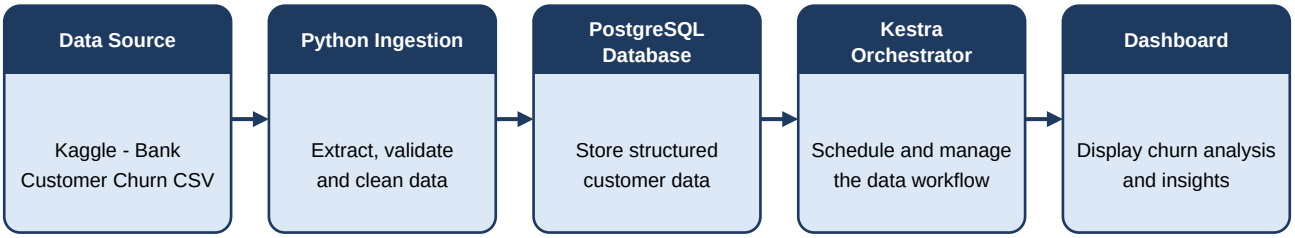
3. Analytical Questions

Q1. Which country (Geography) has the highest churn rate, and how do average balance and credit score differ between churned and retained customers in each country?

Q2. How does churn change by age group and tenure, and which age group is most at risk?

Q3. How do the number of products held and active-member status affect churn probability (e.g. do customers with 3-4 products or inactive members churn more)?

4. Proposed Architecture



PostgreSQL and pgAdmin run as containers on one Docker network (docker-compose).

Flow: The CSV is read by a Python script (pandas + SQLAlchemy/psycopg2), validated and loaded into the PostgreSQL *raw* schema. SQL transforms populate the clean *analytics* schema and views. Kestra runs the ingestion, quality checks and transformations on a schedule, and the dashboard reads the analytics views. PostgreSQL and pgAdmin run in Docker containers on a shared network (de_net) and PostgreSQL data is kept in a persistent volume (pgdata).

5. Expected Database Tables

Schema.Table	Purpose	Key columns
raw.bank_customer_churn	Exact copy of the CSV plus load timestamp	row_number, customer_id, surname, credit_score, geography, gender, age, tenure, balance, num_of_products, has_cr_card, is_active_member, estimated_salary, exited, ingested_at
analytics.customers	Cleaned, typed, deduplicated table with constraints	customer_id (PK), surname, credit_score, geography, gender, age, tenure, balance, num_of_products, has_cr_card (bool), is_active_member (bool), estimated_salary, exited (bool)
analytics.v_churn_by_geography	View for Q1	geography, customers, churn_rate_pct, avg_balance
analytics.v_churn_by_age_group	View for Q2	age_group, customers, churn_rate_pct
analytics.v_churn_by_products_activity	View for Q3	num_of_products, is_active_member, customers, churn_rate_pct

6. Environment & Repository

Docker Compose starts PostgreSQL 16 (with a healthcheck and persistent volume) and pgAdmin 4. Credentials are stored in a local .env file (not committed); .env.example is provided. The GitHub repository follows the required structure: docs/, scripts/, sql/, flows/, docker-compose.yml, .env.example, README.md.
GitHub repo: [paste repo link] | **Video:** [paste unlisted YouTube/Drive link]